



A SOLUTION TO LINE-ROUTING PROBLEMS ON THE CONTINUOUS PLANE

David W. Hightower

Bell Telephone Laboratories, Incorporated
Holmdel, New Jersey

Abstract

This paper discusses a new line-routing algorithm. The algorithm has been programmed in FORTRAN II for the IBM 7094 and in FORTRAN IV for the IBM 360/65. It has given good results when applied to many line-routing problems such as mazes, printed circuit boards, substrates, and PERT diagrams. The main advantages of this algorithm, which is based on the continuous plane, over conventional algorithms based on the discrete plane are twofold:

1. Since the algorithm is based on the continuous plane, there is theoretically no limit to the degree of precision used to describe the position of points. In practice, the only factor restricting the precision is the magnitude of the largest (or smallest) number which may be stored in a computer. As a result, the nodes on a printed circuit board, for example, can be input with mil accuracy. If this feat were to be accomplished by existing methods on a 9x9 inch board, a matrix of 81,000,000 cells would have to be stored (and searched) in the computer.
2. The algorithm stores only line segments; therefore to find a path, only the segments that are currently defined need be investigated. Usually with conventional methods, every cell that lies on every possible minimal path must be investigated. The net result is that this algorithm is much faster than the conventional method.

Introduction

There are several excellent line-routing algorithms in service at this time. Most of them use modified versions of Lee's Algorithm.¹ Some of them use strictly searching techniques to determine if the best path is available; if it is not, the second best path is interrogated, and so forth. Virtually all of these line-routing algorithms require that a matrix or set of squares be stored in the computer. If a particular application requires a very fine grid, and one word of storage is used for each square, then the size of the matrix becomes prohibitive. If parts of words are used, then it may be possible to store a very

large matrix in the computer, but run time is increased far out of proportion to the increased capacity.

The success of some of these methods, however, is indisputable. For some applications the shortest path (or the n^{th} shortest path) is the most desirable and Lee's algorithm yields the shortest path. However, for many applications such as printed circuit wiring, it is not necessary to have the shortest path.

All of the above considerations led the author to believe that a solution to many line-routing problems might lie in a continuous plane approach. Ideally, the solution would require relatively little computer space, would give results that were very close to the results given by Lee's Algorithm, and would reduce run time significantly.

Preliminaries

General

If we are restricted to a Manhattan geometry and it is our task to connect two points, A and B, by a path that does not intersect certain constraints we can see at once that among the line segments that compose the path must be two of the four line segments on which the two points lie. That is, the path must contain a segment of either the horizontal or vertical line on which point A lies, and the same is true of point B.

Consider point A first. We know that the path from A contains either segment ℓ_1 or segment ℓ_2 (see Figure 1). If the line segments ℓ_1 and ℓ_2 are bounded by other line segments, then our choice for the next line segment to be placed in the path must come from the line segments perpendicular to ℓ_1 or from the line segments perpendicular to ℓ_2 which lie between the line segments bounding ℓ_1 and ℓ_2 . In Figure 2, a line segment like a, b, c, d, e, f, or g must appear in the path. A good choice will be one that can escape past the bounds of ℓ_1 and ℓ_2 namely a, b, c, d, or e. Having chosen by some predetermined scheme one of the possibilities, say a, we again may select a set of line segments which must be used to reach a. Continuing in this fashion from both points we will construct a network of line segments. At some point the networks will

TWO STAGES IN MAPWIR LINE-ROUTING ALGORITHMS

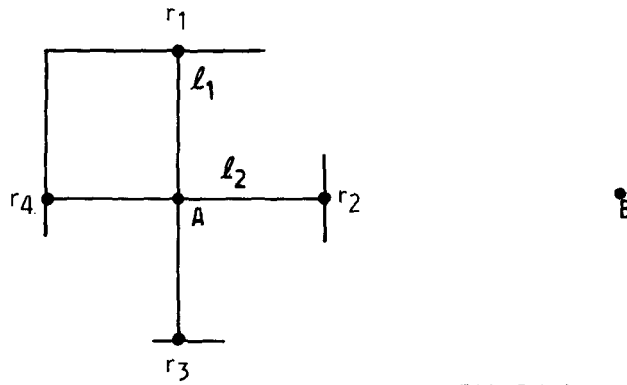


FIGURE 1

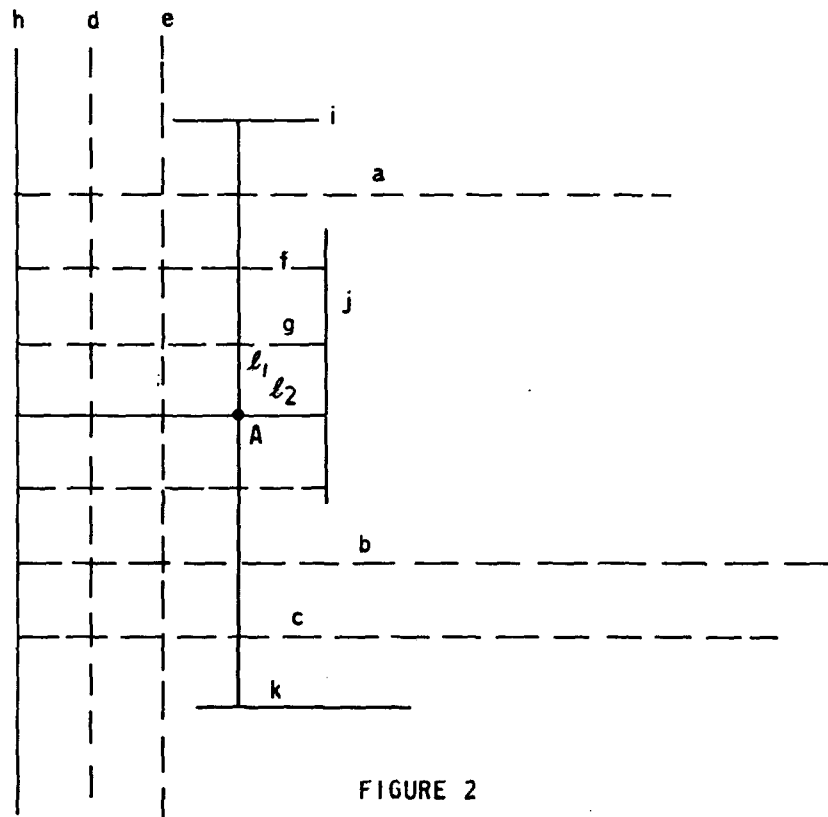


FIGURE 2

overlap, and by tracing backwards from this point to preceding points a path will be established.

The process just described briefly is the principle underlying the algorithms explained in this paper.

Essentially, the main process involves repeatedly constructing lines, looking for intersecting networks, finding escape points, and constructing more lines. The process can be divided into three algorithms: The Escape Algorithm, the Path-Determining Algorithms, and the Main Procedure which repeatedly applies the first two algorithms.

All of the algorithms are first described verbally, then followed by detailed flowcharts. In going through the algorithms, the reader may find it useful to refer to the illustration in Appendix I of the algorithm as applied to the problem of finding a path from A to B in Figure 14. It will also be useful to keep Figure 3 in mind.

Notation

In the notation used throughout this paper, a superscript will refer to the point with which the symbol is associated. A subscript will refer to the orientation of the symbol. For example ℓ_p^h is a single horizontal line segment through the point p, and L_p^v is a set of vertical line segments associated with the point p. L_p^o orientation will refer to the list associated with the point p, containing line segments oriented in the direction of the current value of the orientation flag ("orientation flag" is defined below).

Observe that a point is a line of zero length. As a result, when "line segment" is mentioned, the possibility that the segment has zero length must not be overlooked.

The problem can be stated formally thus: Given a set of nontraversable vertical line segments, C_v , and a set of nontraversable horizontal line segments, C_h , find a taxicab path (only right-angle bends allowed) from point A to point B which does not intersect any member of C_v or C_h .

Definitions (see Figure 3 for illustrations)

D1: A line segment ℓ (vertical or horizontal) is said to cover a point, p, if the perpendicular from p to the line of which ℓ is a segment intersects ℓ .

D2: A line segment, ℓ in C_h , is said to be a horizontal cover of the point p if ℓ covers p, and if no member of C_h which covers p lies between p and ℓ . There are two such line segments for each point p, one above and one below p.

D3: A vertical cover is defined similar to a horizontal cover.

D4: The vertical line segment passing through p and bounded by the horizontal covers of p will be called the vertical escape line.

D5: The horizontal line segment passing through p and bounded by the vertical covers of p will be called the horizontal escape line.

D6: A point, e, on the vertical (horizontal) escape line of the point p is said to be an escape point if it is not covered by a vertical (horizontal) cover of p and is not covered by any member of C_v (C_h) which lies between p and the vertical (horizontal) cover.

D7: A unit is the minimum distance allowed between line segments.

D8: The object point is the escape point that is currently being processed. The current object point is the escape point that was last found.

D9: The target point is the point that must be reached from the object point.

Lists

P: List of points which determine a path.

L_h^o : List of horizontal line segments associated with the object point and ordered on the ordinates.

L_v^o : List of vertical line segments associated with the object point ordered on the abscissas.

L_e^o : List of escape points associated with the object point.

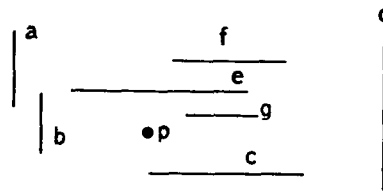
There is also a set of lists like those above associated with the target point denoted by a super t.

Flags

Intersect: The intersect flag is set when an escape line just constructed intersects any member of L_h^t or L_v^t .

ILLUSTRATIONS OF DEFINITIONS

1. COVER (VERB)

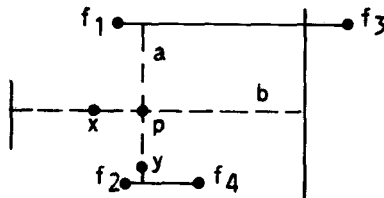


LINE SEGMENTS b,c,d,e COVER p.
LINE SEGMENTS a,f AND g DO NOT.

2. COVER (NOUN)

IN THE ABOVE DRAWING,
LINE SEGMENTS e AND c ARE HORIZONTAL COVERS
LINE SEGMENTS b AND d ARE VERTICAL COVERS

3. ESCAPE LINES



LINE SEGMENT a IS THE VERTICAL ESCAPE LINE.
LINE SEGMENT b IS THE HORIZONTAL ESCAPE LINE.
POINT x IS A VERTICAL ESCAPE POINT.
POINT y IS A HORIZONTAL ESCAPE POINT.

FIGURE 3

Orientation: The orientation flag is set to "HORIZONTAL" or "VERTICAL" depending on the orientation of the escape line to be constructed at the last escape point. There is an orientation flag associated with each original point.

No-escape: The no-escape flag will be set if an escape point cannot be found. There will be a no-escape flag for each point.

Escape Algorithm

Main

In P2 below, L_x^t refers to $L_{\text{HORIZONTAL}}^t$ if $x = \text{"HORIZONTAL"}$ and L_{VERTICAL}^t if $x = \text{"VERTICAL"}$.

- P1: Examine the orientation flag and construct the appropriate escape line through the object point. Enter the line in $L_{\text{Orientation}}$.
- P2: Let x be the complement of the orientation flag (i.e., if the flag is "HORIZONTAL," x will be "VERTICAL"), and examine L_x^t to see if the new escape line intersects any entry in the list. If so, set the intersect flag and continue at P5. If not, continue at P3.
- P3: Attempt to find an escape point by Escape Process I. If this fails apply Escape Process II. If this fails set the no-escape flag, and stop; otherwise continue at P4.
- P4: Push the escape point onto L_e^0 . Stop.
- P5: Let (x,y) be the point of intersection. Stop.

Escape Process I

The definition of an escape point in the context of this process is the same as definition D6. Let Z be the object point.

Let f_1, f_2, f_3, f_4 be the extremities of a cover, horizontal or vertical (see Figure 3.3). Let d be the Euclidean distance function:

$$d((x_1, y_1), (x_2, y_2)) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Associate with (f_1, f_2, f_3, f_4) the quadruple $(f_{i_1}, f_{i_2}, f_{i_3}, f_{i_4})$ such that

$$d(f_{i_m}, Z) \leq d(f_{i_n}, Z), \quad m < n$$

In other words, $(f_{i_1}, f_{i_2}, f_{i_3}, f_{i_4})$ is a permutation of (f_1, f_2, f_3, f_4) . In Figure 3.3 $(f_{i_1}, f_{i_2}, f_{i_3}, f_{i_4}) = (f_2, f_4, f_1, f_3)$.

P1: Let a vector $(f_{i_1}, f_{i_2}, f_{i_3}, f_{i_4})$ be associated with the horizontal cover. Starting with $m = 1$, take the point whose abscissa is one unit away from the abscissa of f_{i_m} ("away" means "away from Z ") and whose ordinate equals the ordinate of Z . Test to see if this is an escape point lying on a vertical escape line not previously used. Continue incrementing i until either $i = 5$ or an escape point is found. If one is found, set the orientation flag to "VERTICAL" and stop. If $i = 5$, continue at P2.

P2: Let the vector $(f_{i_1}, f_{i_2}, f_{i_3}, f_{i_4})$ be associated with the vertical cover. Starting with $m = 1$, take the point whose ordinate is one unit away from the ordinate of f_{i_m} and whose abscissa equals the abscissa of Z . Test to see if it is an escape point lying on a horizontal escape line not previously used. Continue incrementing i until $i = 5$ or an escape point is found. If one is found, set the orientation flag to "HORIZONTAL," and stop. If $i = 5$, there is no escape by this process.

Escape Process II

The definition of an escape point in the context of this process is as follows: A point is said to be a Process II escape point if it has a Process I escape point on either the horizontal or the vertical escape line.

Let r_1 be the intersection of the vertical escape line and the horizontal cover above Z ; r_2 , the intersection of the horizontal escape line and the vertical cover to the right of Z ; r_3 , the intersection of the vertical escape line and the horizontal cover below Z ; and r_4 , the intersection of the horizontal escape line and the vertical cover to the left of Z (see Figure 1). Starting with $i = 1$, take r_i to be one unit closer to Z than the previous r_i . Construct the appropriate escape line through r_i (the other escape line of r_i already exists as an escape line of Z). If the intersect flag is set, push r_i onto L_e^0 and stop. If the flag is not set, try to find a Process I escape point on either of the two escape lines as outlined in the Escape Algorithm. If a Process I escape point cannot be found, let i be the next number in the cycle (1,2,3,4)

FLOWCHART OF ESCAPE ALGORITHM

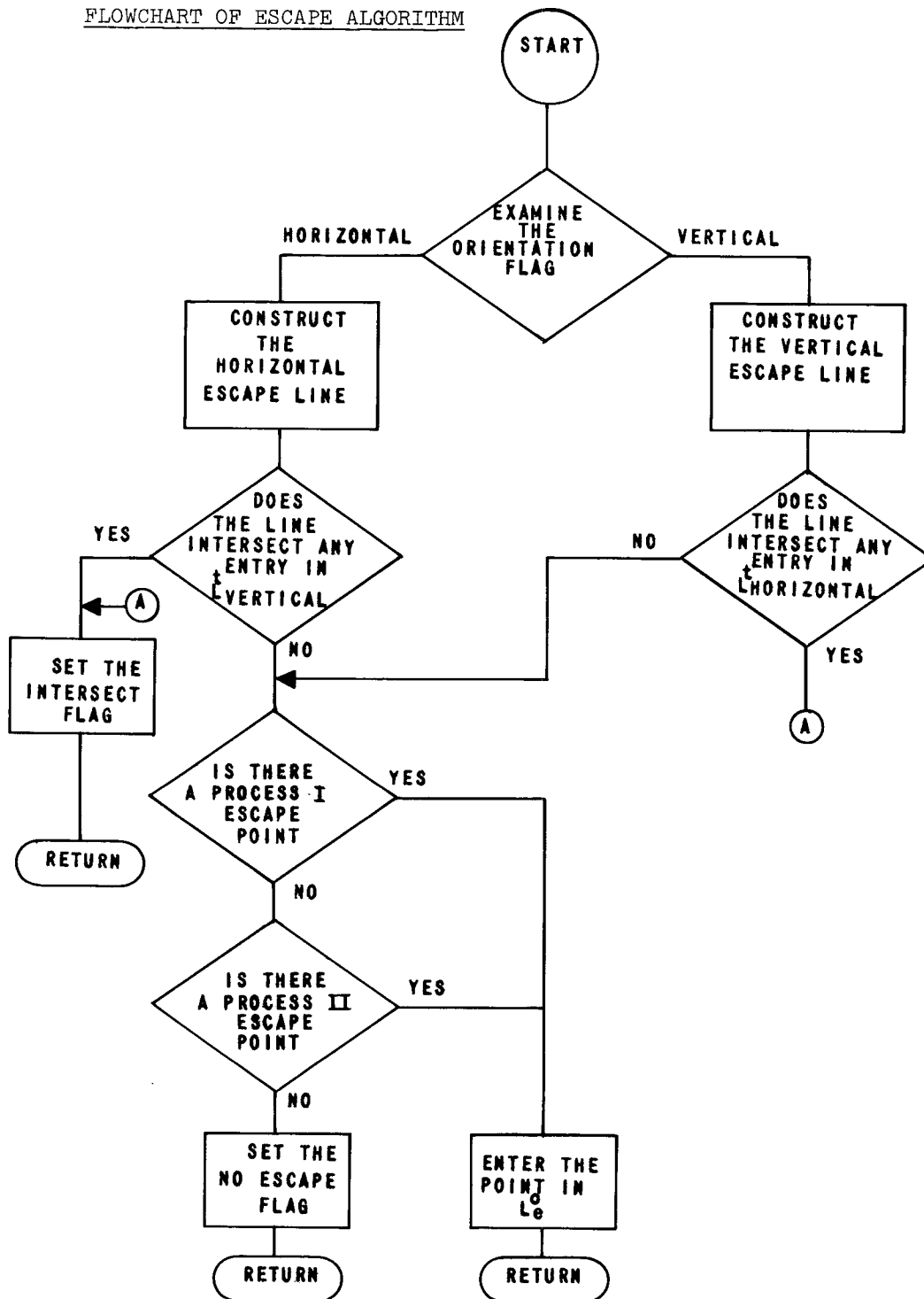


FIGURE 4

ESCAPE PROCESS I

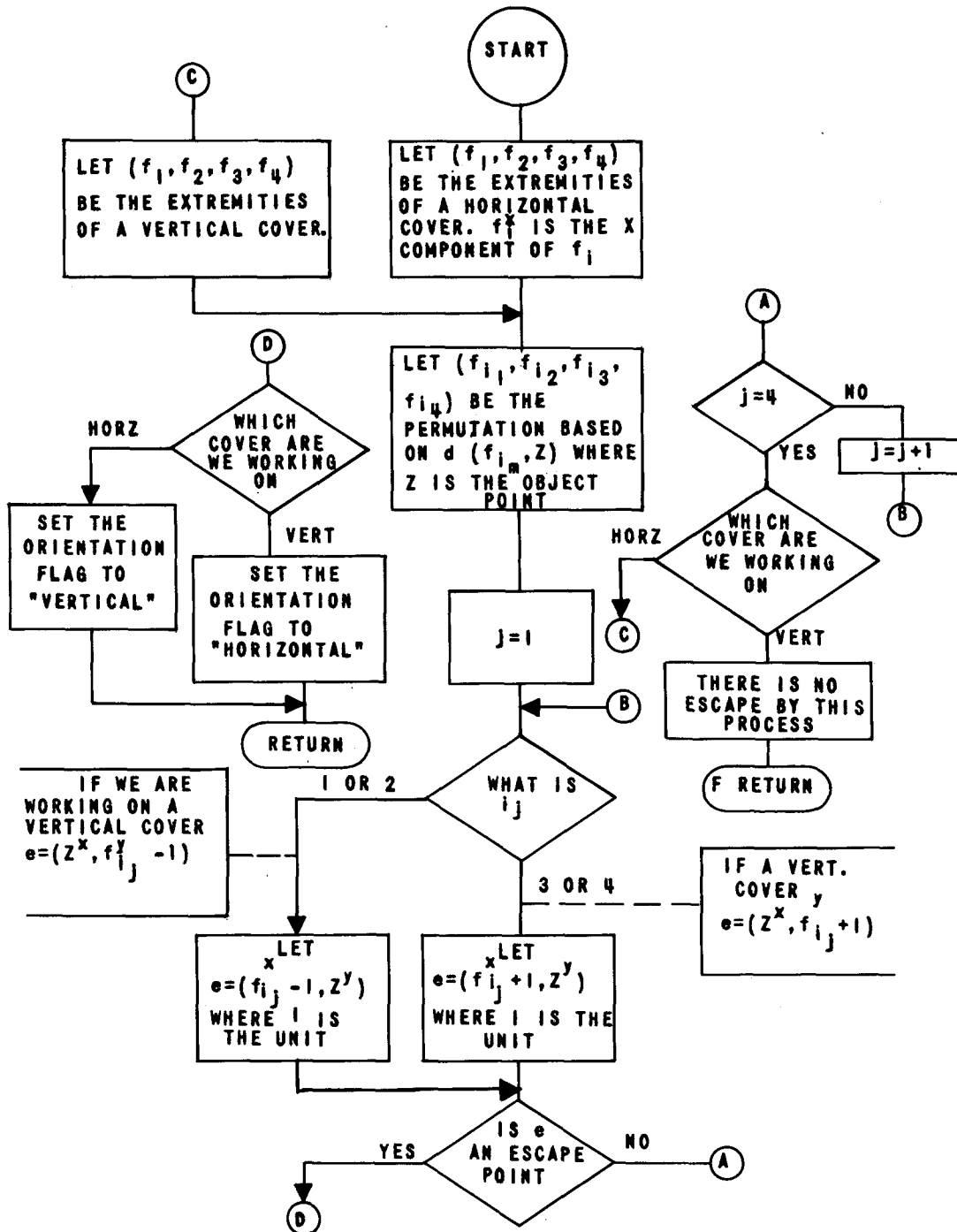


FIGURE 5

ESCAPE PROCESS II

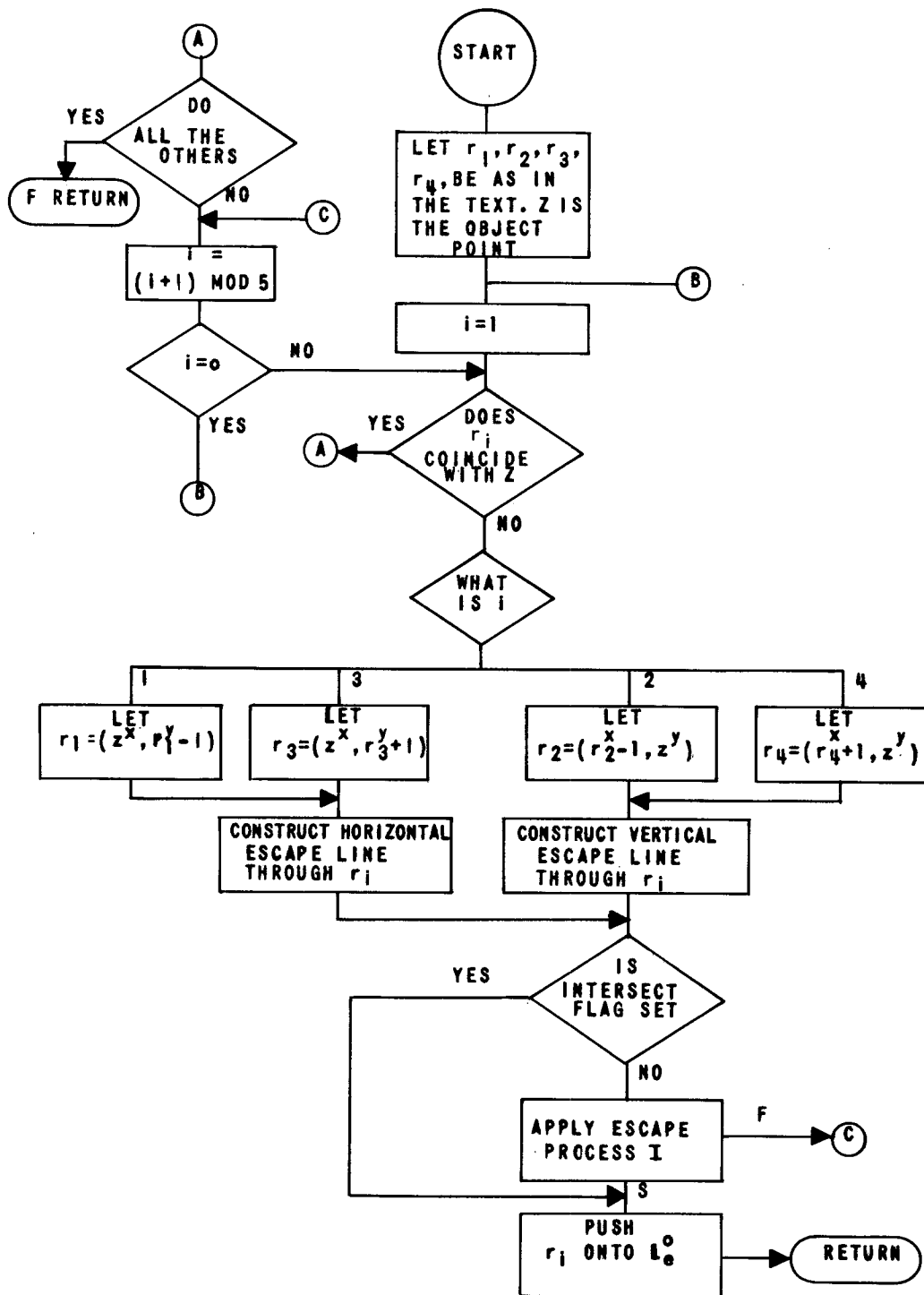


FIGURE 6

and continue as above until either a Process II escape point is found or r_1, r_2, r_3, r_4 all coincide with Z , in which case this process fails. If an escape point is found, push r_1 onto L_e^0 .

The Process I escape point now becomes the escape point to be dealt with in P4 of the Escape Algorithm.

Path Determining Algorithms

This set of algorithms is necessary because the preceding algorithms, lacking a priori knowledge of the most efficient choice of escape points, tend to generate more points than are necessary for a path. The "extra" points are sometimes rather badly scattered about. It is the purpose of the path-determining algorithms to discard those points which are superfluous.

First Refinement Algorithm

This algorithm operates on an ordered list of points of the form $P = \{A, p_1, p_2, \dots, p_n\}$, where the $\{p_i\}$ are escape points associated with the point A . Let (x, y) be the point of intersection found in P2 of the main Escape Algorithm. There is also a line, l , passing through (x, y) and p_n which is either the intersecting or intersected line. The list defines a path from A to (x, y) .

The first refinement is to isolate those points which are corners in the path.

Theorem: If $p_k, p_{k+1}, \dots, p_{k+m}$ is a segment in the above path such that p_k and p_{k+m} lie on the same escape line, p_{k-1} does not lie on that line, and $\{p_{k+m}, \dots, p_n\}$ has been "first refined" (meaning that each point in $\{p_{k+m}, \dots, p_n\}$ represents a corner, i.e., no three successive points lie on the same line), then $p_{k+1}, \dots, p_{k+m-1}$ can be deleted and the path will be preserved.

The need for this theorem is made clear when one understands that the First Refinement algorithm accomplishes the task of isolating those points which represent corners by starting with a particular escape point, constructing the appropriate escape line through it, finding the first escape point lying on that line, and deleting everything in between. It is not obvious that critical parts of the path have not been deleted. It should be noted that no information about the relative position of points can be obtained from their order in the list. Thus if p_k, p_{k+1}, p_{k+2} all lie on the same escape line, it is not necessarily true that p_{k+1} lies between p_k and p_{k+2} .

Proof: Since p_{k-1} is not in the segment, $\{p_{k-1}, p_k\}$ must define a line segment perpendicular to the segment $\{p_k, \dots, p_{k+m}\}$. Similarly, $\{p_{k+m}, p_{k+m+1}\}$ must define a segment perpendicular to $\{p_k, \dots, p_{k+m}\}$, since $\{p_{k+m}, \dots, p_n\}$ has been "first refined." The question now is, can there be any i such that $1 < i < k - 1$ or $k + m + 1 < i < n$ and p_i lies on $\{p_k, \dots, p_{k+m}\}$? If there is such an i , p_i must be an escape point, which means the escape line $\{p_k, \dots, p_{k+m}\}$ must have been constructed through it. But the Escape Algorithm does not choose escape points that lie on already existing escape lines. Thus, if $1 < i < k - 1$, the escape line corresponding to $\{p_k, \dots, p_{k+m}\}$ must have already been constructed when it became necessary to find escape point p_k ; therefore, p_k could not have been chosen - a contradiction.

If $k + m + 1 < i < n$, then the escape line $\{p_k, \dots, p_{k+m}\}$ must have already been constructed when it became necessary to find escape point p_i ; thus, p_i could not have been chosen - a contradiction. Therefore, such an i does not exist.

If $k < j < k + m$, then p_j must lie on the segment $\{p_k, \dots, p_{k+m}\}$. If we suppose otherwise it leads to the contradiction that p_{k+m} could not have been chosen as an escape point since it would lie on a previously chosen escape line.

Thus it has been established that all points in $\{p_k, \dots, p_{k+m}\}$ lie on the same line segment, and that all points in P that lie on the escape line of which $\{p_k, \dots, p_{k+m}\}$ is a segment, lie on $\{p_k, \dots, p_{k+m}\}$. The truth of the theorem should now be evident.

This theorem suggests the following algorithm: Let L be a list, $k = 1$ (1 is the line segment mentioned in the opening paragraph in this section).

- P1: Find the least i such that p_i (or A) lies on k .
- P2: Delete p_i and all points after it. Push p_i onto L .
- P3: Find the escape line through p_i perpendicular to k . Let $k =$ this line.
- P4: Is P exhausted? If so stop; if not continue at P1.

Second Refinement Algorithm

After the first improvement, every point in $L = \{p_1, p_2, \dots, p_n\}$ represents a corner in the path. However, there may still be parts of the path that are superfluous.

FIRST REFINEMENT ALGORITHM

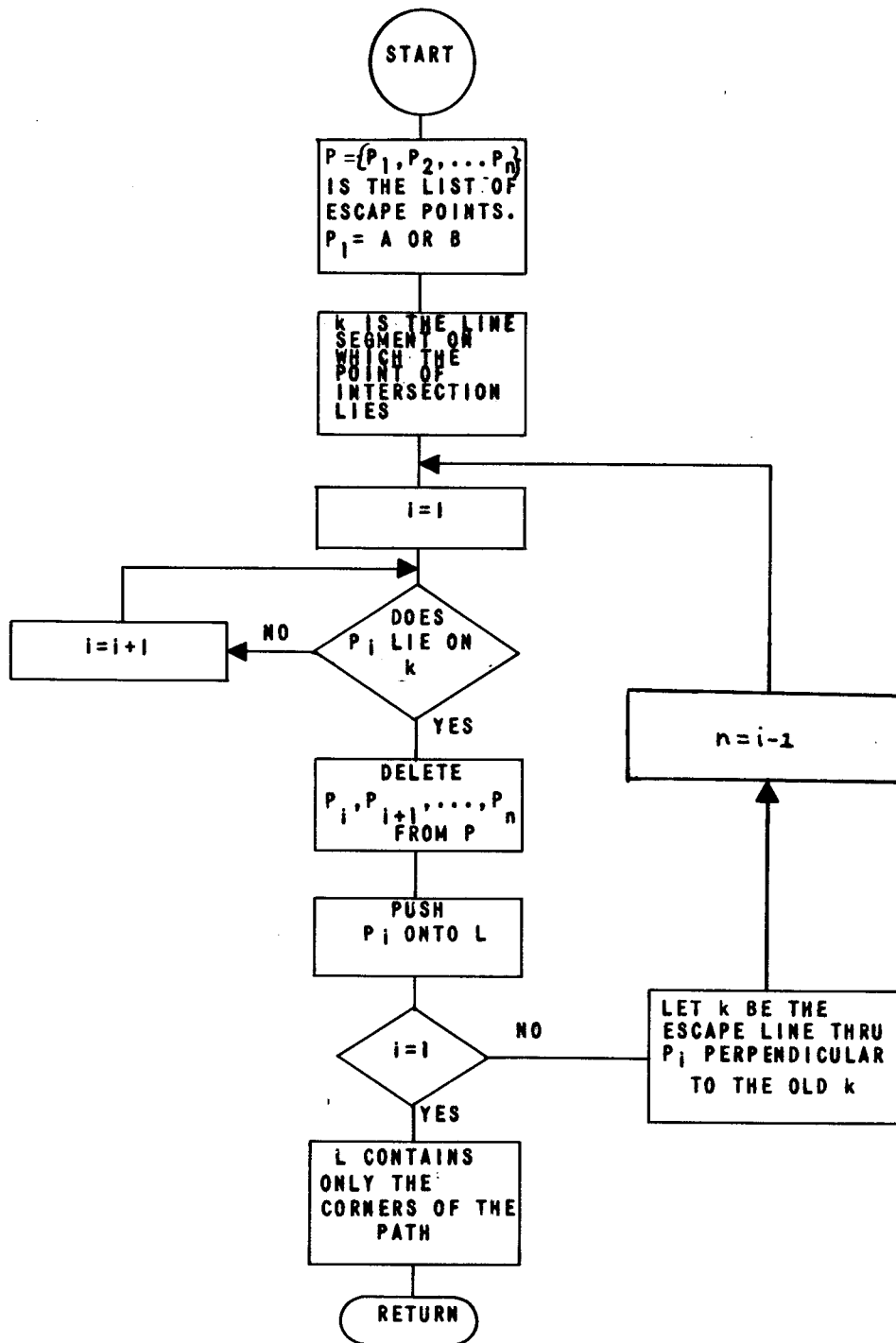
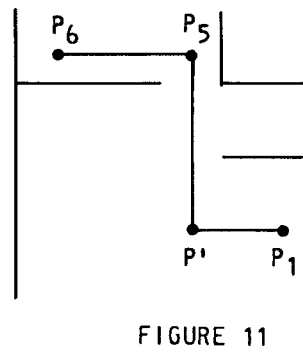
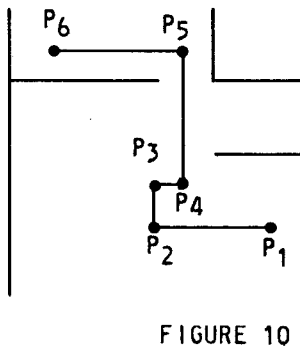
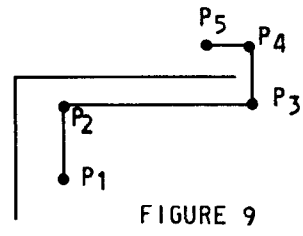
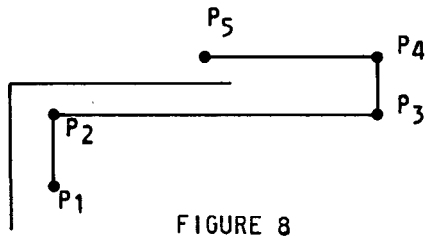


FIGURE 7

EXAMPLES OF THE SECOND IMPROVEMENT



FLOWCHART OF THE SECOND IMPROVEMENT

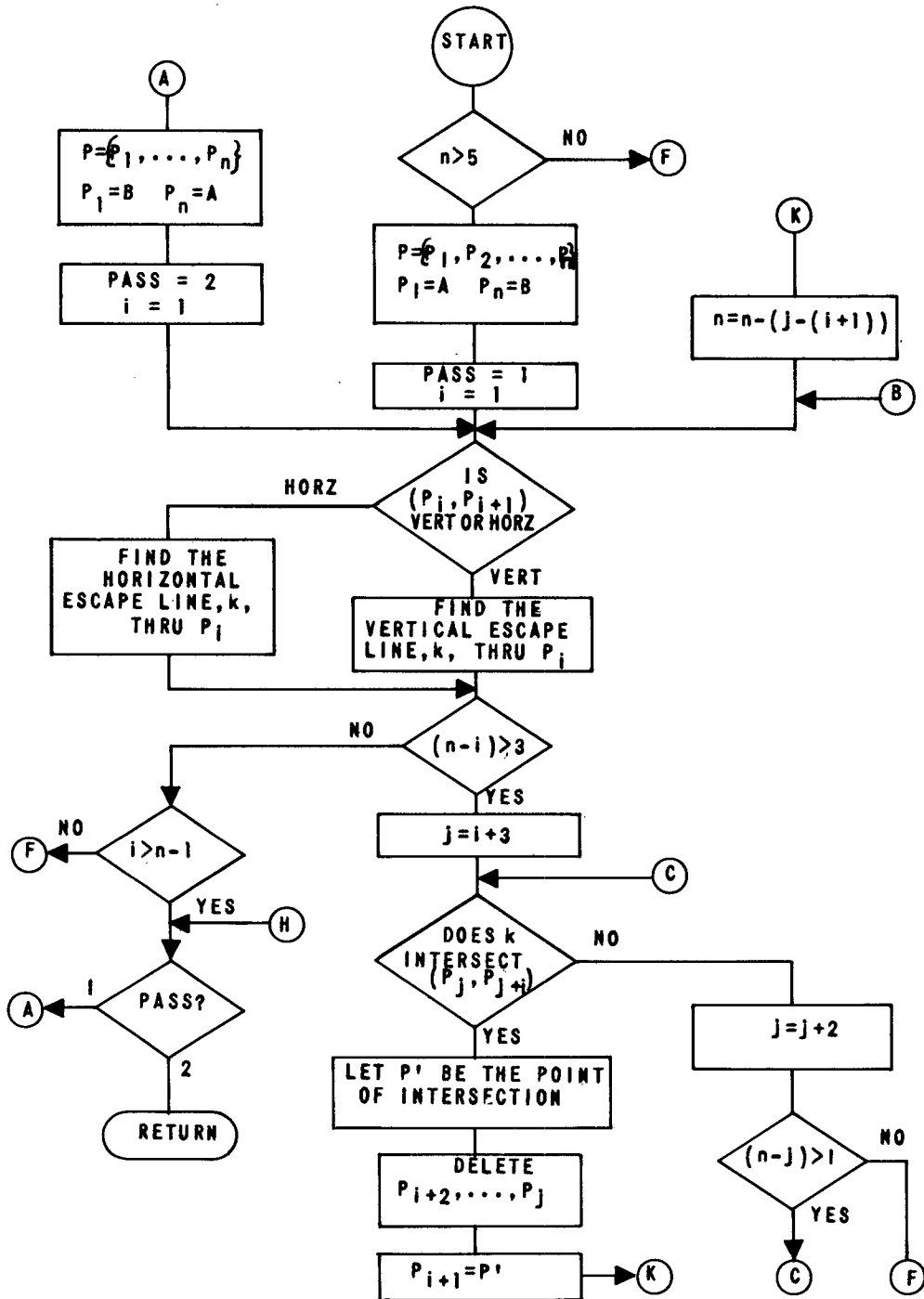


FIGURE 12

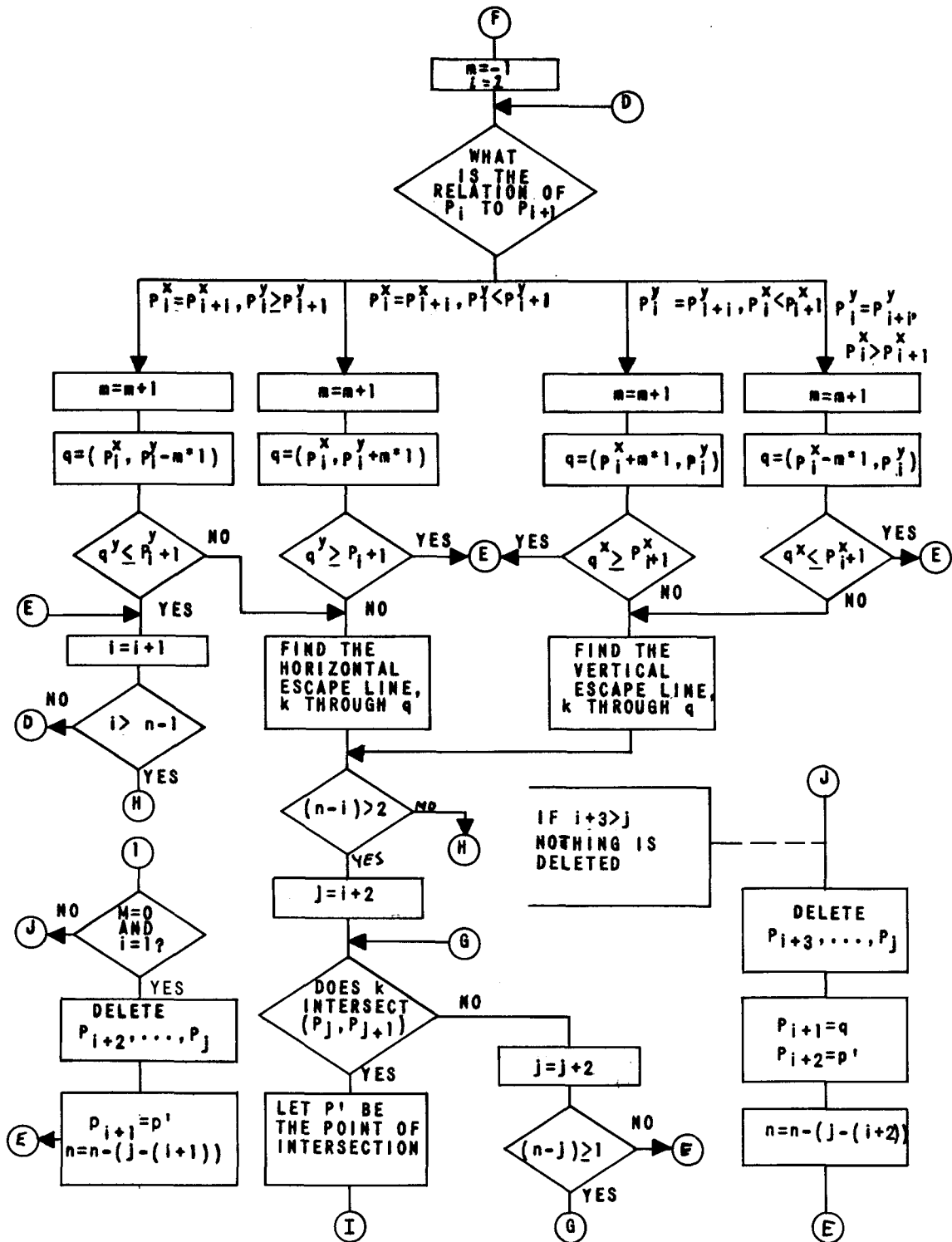


FIGURE 12 CONT.

As an example consider Figure 8. The path shown is much longer than it need be. (p_2, p_3) and (p_4, p_5) could be shortened considerably as shown in Figure 9.

As another example, in Figure 10 it is clear that if (p_5, p_4) were extended to meet (p_1, p_2) , then p_4 , p_3 , and p_2 could be replaced with the intersection of (p_5, p_4) and (p_1, p_2) and the shorter path shown in Figure 11 would result.

To effect the improvement in the second example (Figure 10), successively extend each segment of the path until it is stopped by a constraint. If it intersects any other segment, (p_k, p_{k+1}) , replace all points between the originating point of the extended segment and p_{k+1} by the point of intersection.

To effect the improvement in the first example, successively consider each segment in the path and construct lines perpendicular to the segment at unit intervals along it. The lines are constructed starting at one end of the segment and are bounded by constraints. As each of the lines is constructed a check is made to see if it intersects any other segment in the path. If it does intersect some other segment, (p_k, p_{k+1}) , then all points between the originating point of the segment and p_{k+1} are replaced by the point of intersection of the constructed line and (p_k, p_{k+1}) and by the point on the original segment through which the line was constructed.

A precise statement of the algorithm is given in the flowchart of Figure 12.

Main Procedure

- P0: Push A onto L_e^A and B onto L_e^B .
- P1: Test the no-escape flag associated with point A. If it is set, test the no-escape flag from point B. If it is also set then the algorithm has failed to find a path. If the B flag is not set, continue at P2. If the A flag is not set, let the last entry in L_e^A be the object point and point B the target point. Apply the Escape Algorithm. If the intersect flag is set, continue at P3, otherwise continue at P2.
- P2: In P1 change A to B, B to A, and P2 to P1 and apply the results.
- P3: Apply the First Refinement algorithm to L_e^A and L_e^B . Now $L_e^A = \{A, a_1, a_2, \dots, a_m\}$ and $L_e^B = \{B, b_1, b_2, \dots, b_n\}$

(x, y) is the point of intersection. Let $P = \{A, a_1, a_2, \dots, a_m, (x, y), b_n, b_{n-1}, \dots, b_1, B\}$. Apply the Second Improvement algorithm. Then P contains the path from A to B in terms of the coordinates of each bend in the path.

Advantages and Disadvantages

The line-routing algorithm described in this paper can be used in applications where the shortest path is not the paramount objective, or where the time factor outweighs the path length criterion.

The main advantages of the algorithm are its speed, flexibility, and the fact that refinement is not bounded by storage considerations.

The main disadvantages are that it does not always yield the shortest path, and that it is possible that the algorithm will not find a path when one exists. To overcome the first disadvantage, it is possible to make several successive run-time redefinitions of certain of the algorithms, each of which would produce a different path; the shortest of these paths could then be selected.

The second disadvantage is due largely to the fact that the algorithm does not use the same escape line more than once. This means that if the algorithm used an escape line to get into a box that had a very narrow opening, it could not get out again. In all of the experiments that were run, a path was always found when one existed.

Application

The program can be used in four modes: The first completes as many of the connections as possible without allowing crossovers. The second completes as many connections as possible without allowing crossovers and then completes as many of the remaining connections as possible allowing crossovers. The third allows crossovers from the beginning. The fourth mode completes as many connections as possible. The remaining connections are made on a clean surface. This mode simulates wiring on a two-sided printed circuit board.

These four modes are implementations of the basic algorithm described herein.

Conclusion

The algorithm described in this memorandum has proven successful when applied to the type of problems shown in the Appendix.

FLOWCHART OF THE MAIN PROCEDURE

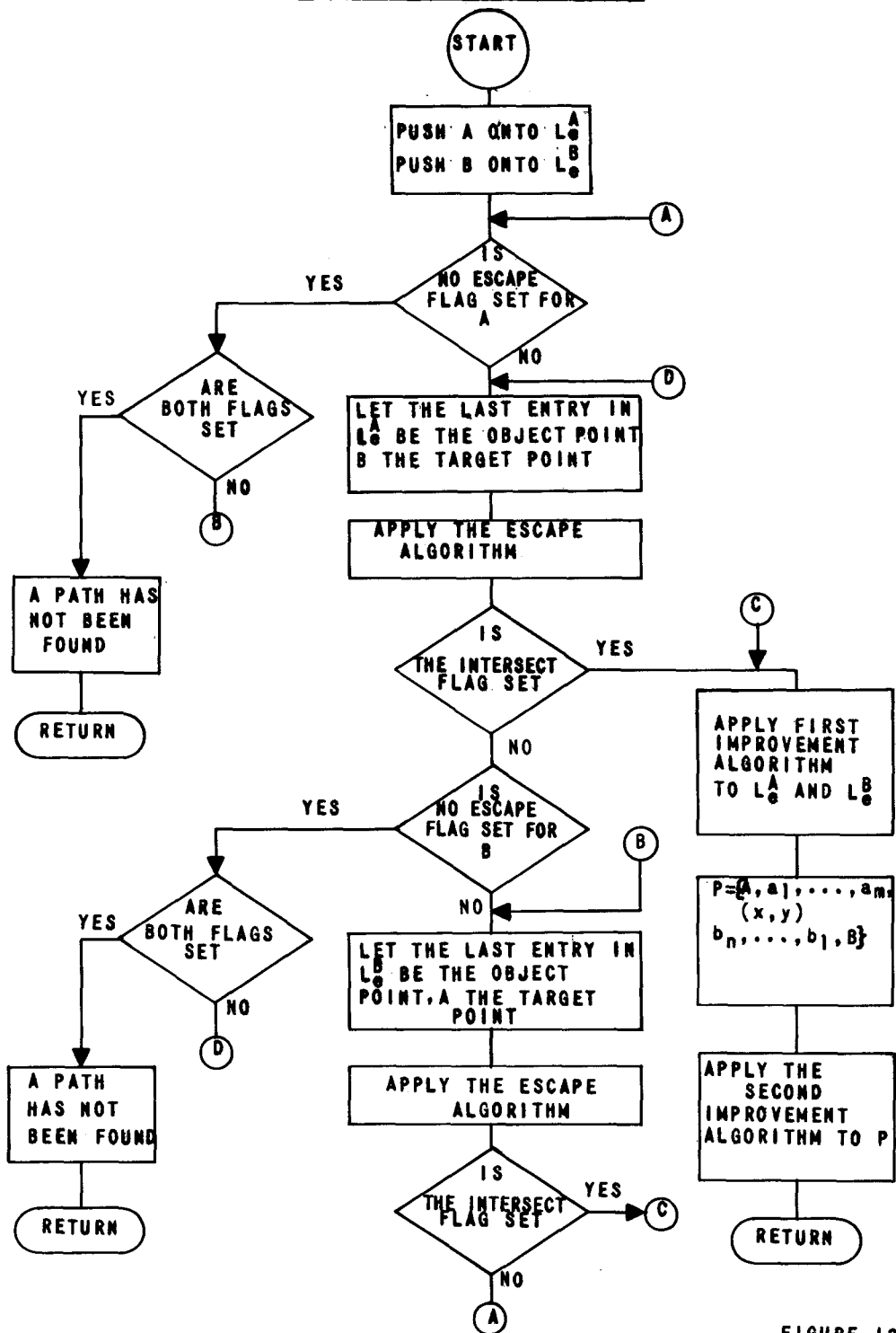


FIGURE 13

PROBLEM ONE

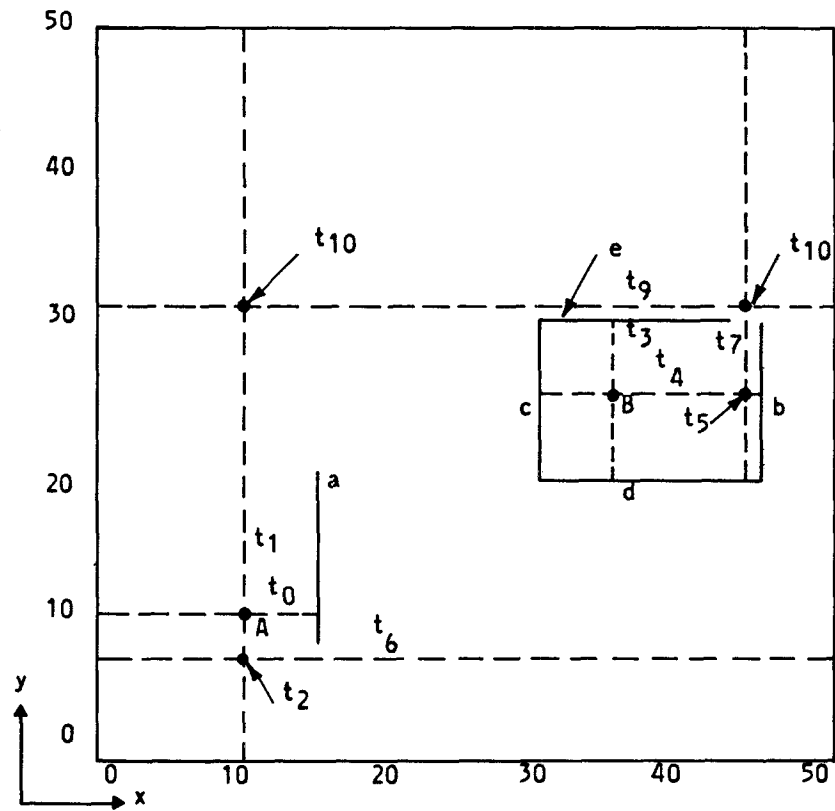


FIGURE 14

The speed of the algorithm as programmed in FORTRAN IV for the IBM 360/65 makes the algorithm useful for applications where the conventional methods are too sluggish or otherwise impractical, such as interactive graphical display applications.

References

1. C. Y. Lee, "An Algorithm for Path Connections and Its Applications," IRE Transactions on Electronic Computers; pp. 346 through 365, September, 1961.

Appendix A

In Figure 14, page 16, the task is to find a path from A to B without intersecting a, b, c, d, or e.

Main Procedure

P1: The A no-escape flag is not set, so let A be the object point, B the target point.

The intersect flag is not set.

P2: The B no-escape flag is not set, so let B be the object point, A the target point.

Escape Algorithm

P1: The orientation flag is set to BOTH. Construct t_0 and t_1 . Enter t_1 in L_h^A , t_0 in L_h^A .

P2: No intersection

P3: Escape Process I locates t_2 as the escape point. The A orientation flag is set to HORIZONTAL.

P4: Push t_2 onto L_e^A .

RETURN

P1: The orientation flag is set to BOTH. Construct t_3 and t_4 . Enter t_3 in L_v^B and t_4 in L_h^B .

P2: t_3 does not intersect t_0 , and t_4 does not intersect t_1 . Therefore, there is no intersection.

P3: Escape Process I locates t_5 as the escape point. The B orientation flag is set to VERTICAL.

Main Procedure

The intersect flag is not set.

P1: The A no-escape flag is not set so let t_2 be the object point, and B the target point.

The intersect flag is not set.

P2: The B no-escape flag is not set so let t_5 be the object point, and A the target point.

The intersect flag is not set.

P1: The A no-escape flag is set. The B no-escape flag is not set.

P2: Let t_8 be the object point, and A the target point.

Escape Algorithm

P4: Push t_5 onto L_e^B .
RETURN

P1: The orientation flag is set to horizontal. Construct t_6 . Enter t_6 in L_h^A .

P2: t_6 does not intersect t_3 . Therefore there is no intersection.

P3: Since t_1 and t_6 extend from boundary to boundary, the no-escape flag is set.

RETURN

P1: The orientation flag is set to VERTICAL. Construct t_7 . Enter t_7 in L_v^B .

P2: t_7 does not intersect t_6 or t_0 . Therefore, there is no intersection.

P3: Escape Process I locates t_8 as the escape point. The B orientation flag is set to HORIZONTAL.

P4: Push t_8 onto L_e^B .

P1: The orientation flag is set to HORIZONTAL. Construct t_9 . Enter t_9 in L_h^B .

P2: t_9 intersects t_1 . Set the intersection flag. The point of intersection is $(t_1, t_9) = t_{10}$

RETURN

Main Procedure

The intersect flag is set.

P3: Apply the Path-Determining Algorithms to:

$L_e^A = \{A, t_2\}$,
 $\ell = t_1$

And to:

$L_e^B = \{B, t_5, t_8\}$
 $\ell = t_9$

$P = \{A, t_{10}, t_8, t_5, B\}$

Path-Determining Algorithms

P1: $k = t_1$, A lies on k .

P2: Delete A and t_2 .
Push A onto L.

P3: $k = t_0$

P4: We are finished.

P1: $k = t_9$, t_8 lies on k .

P2: Delete t_8 . Push t_8 onto L.

P3: $k = t_7$.

P4: $P = \{B, t_5\}$.

P1: t_5 lies on k .

P2: Delete t_5 . Push t_5 onto L.

P3: $k = t_4$.

P4: $P = \{B\}$.

P1: B lies on k .

P2: Delete B. Push B onto L.

P3: $k = t_3$.

P4: We are finished.

Appendix B

The following examples demonstrate the speed and versatility of the algorithm.

The times shown reflect only the time required to find the path or paths; I/O time is not included. All drawings were produced by the Electronics Associated Inc. 3500 Series Dataplotter.

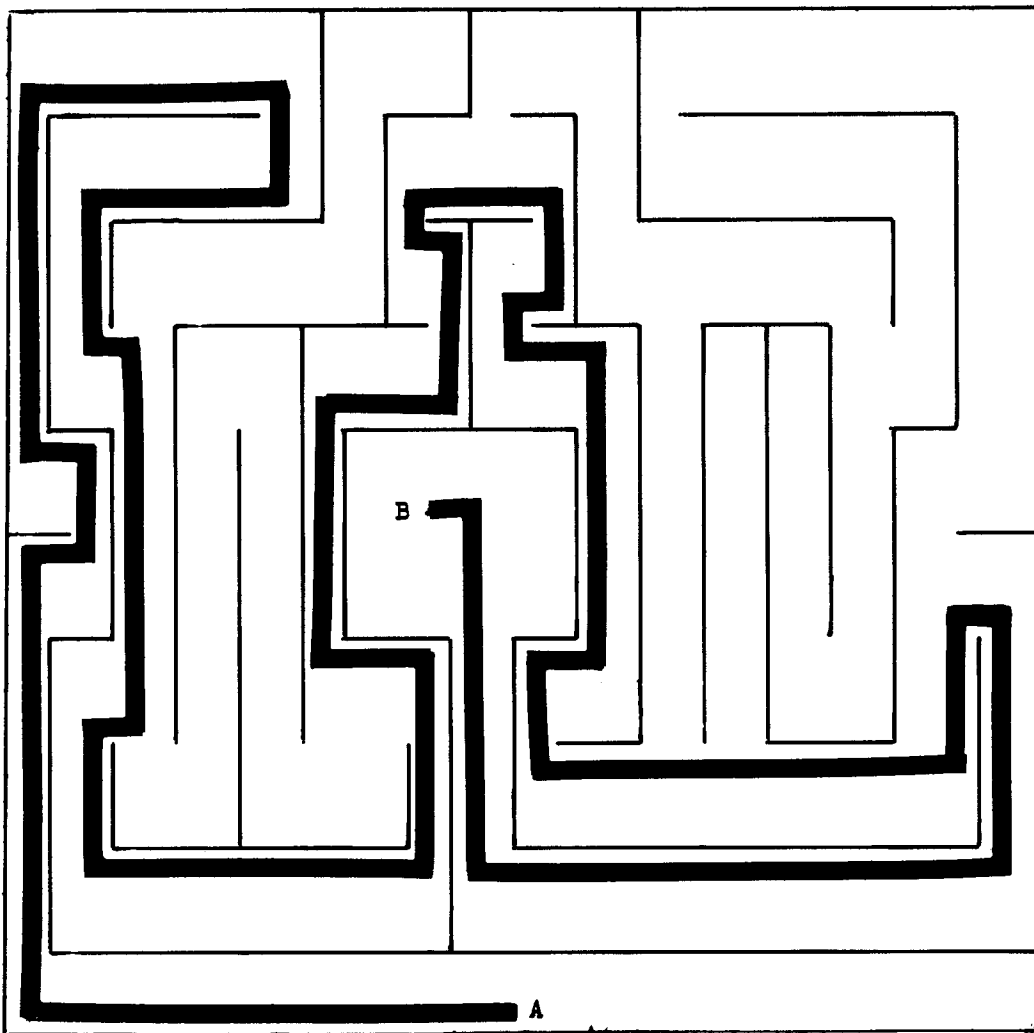
For the printed circuit boards every path that is found is entered in C_v and C_h so that crossovers will not occur. For the PERT diagram only the corners of the path are entered in C_v and C_h . As a result, crossovers are allowed but no two paths can overlap.

For the printed circuit boards, the order in which the nets are taken from the input list of nets to be wired can be any function the user supplies. The most favorable results were achieved when nets were taken in increasing order of $2\Delta Y + \Delta X$, where ΔY is the magnitude of the difference of the Y coordinates of the points to be connected, and ΔX is the magnitude of the difference of the X coordinates. This function favors small changes in the Y; thus given two sets of points, $(Z1, Z2)$ and $(Z3, Z4)$, if $d(Z1, Z2) = d(Z3, Z4)$ (see page 5) then the set with the smallest change in Y will be considered first.

The examples from B-2 to B-6 were done on the IBM 7094. B-7 was done on the IBM 360/65.

SOLUTION TO HAMPTON COURT MAZE

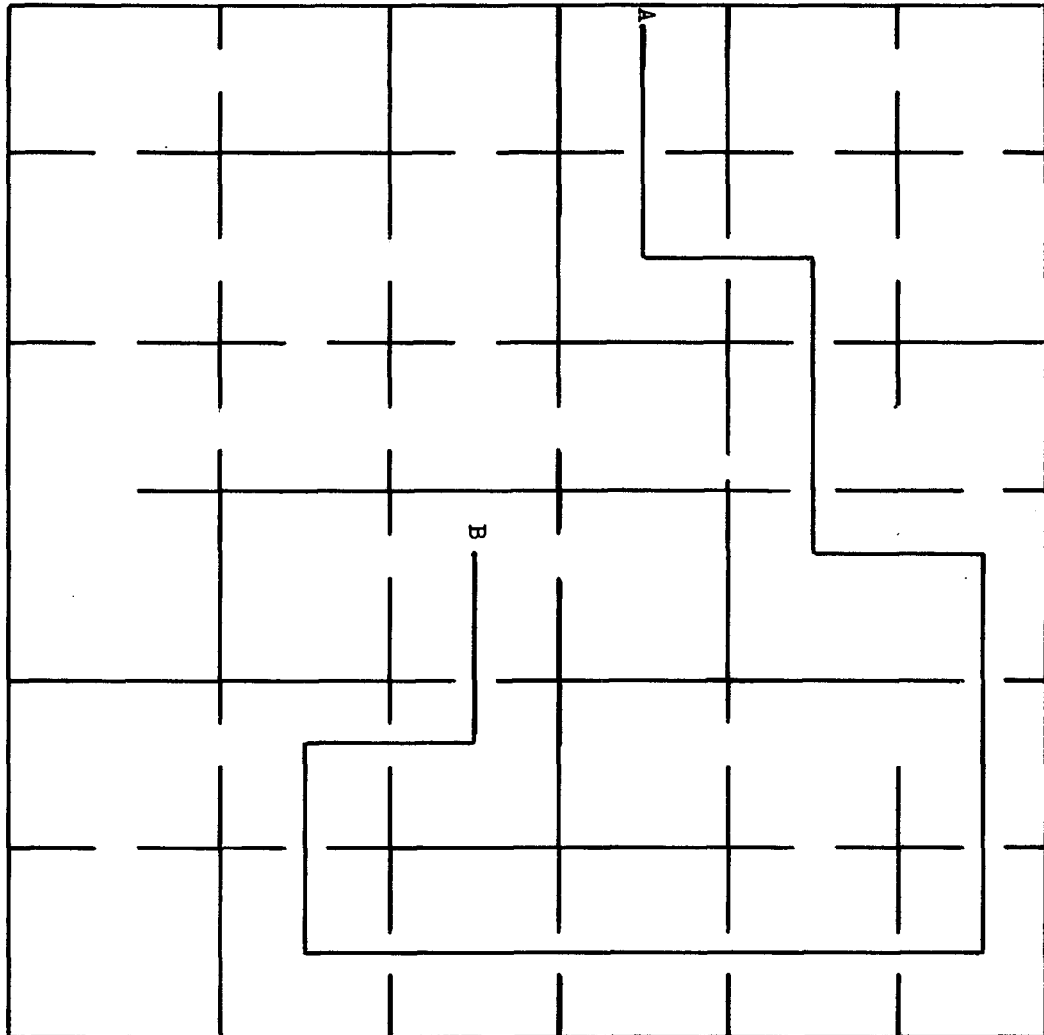
FOUND PATH FROM A TO B
TOTAL TIME .0005



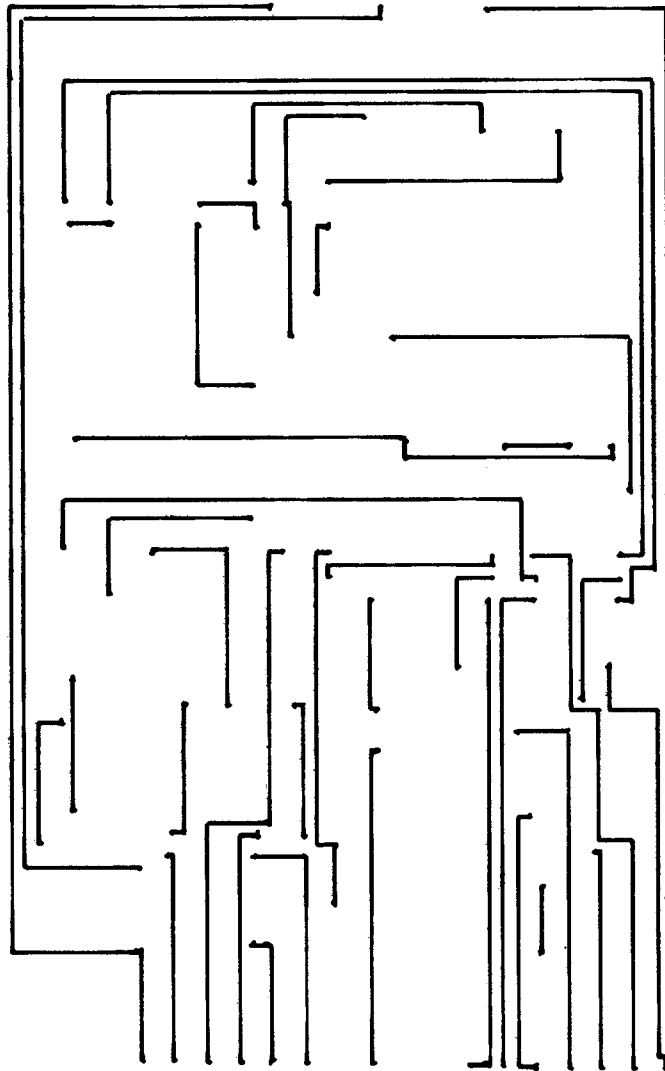
SOLUTION TO MAZE

Found Path From A to B

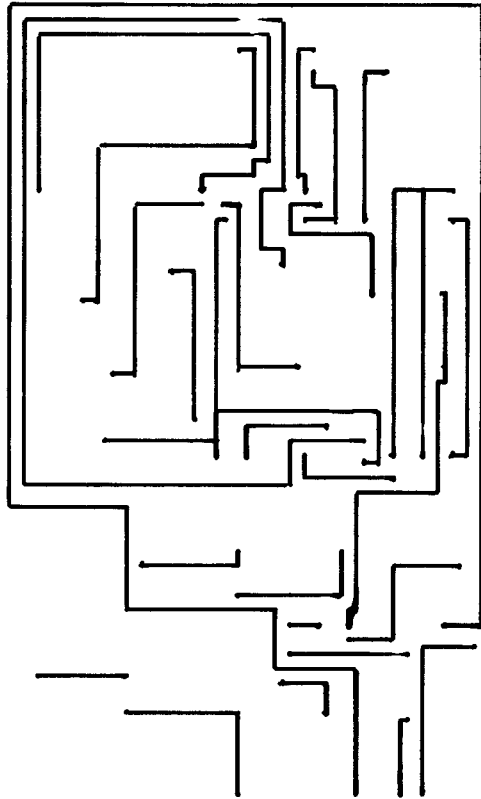
Total Time .00017



44PAIRS OF POINTS
TOTAL TIME .0014 HRS



PRINTED CIRCUIT BOARD - I

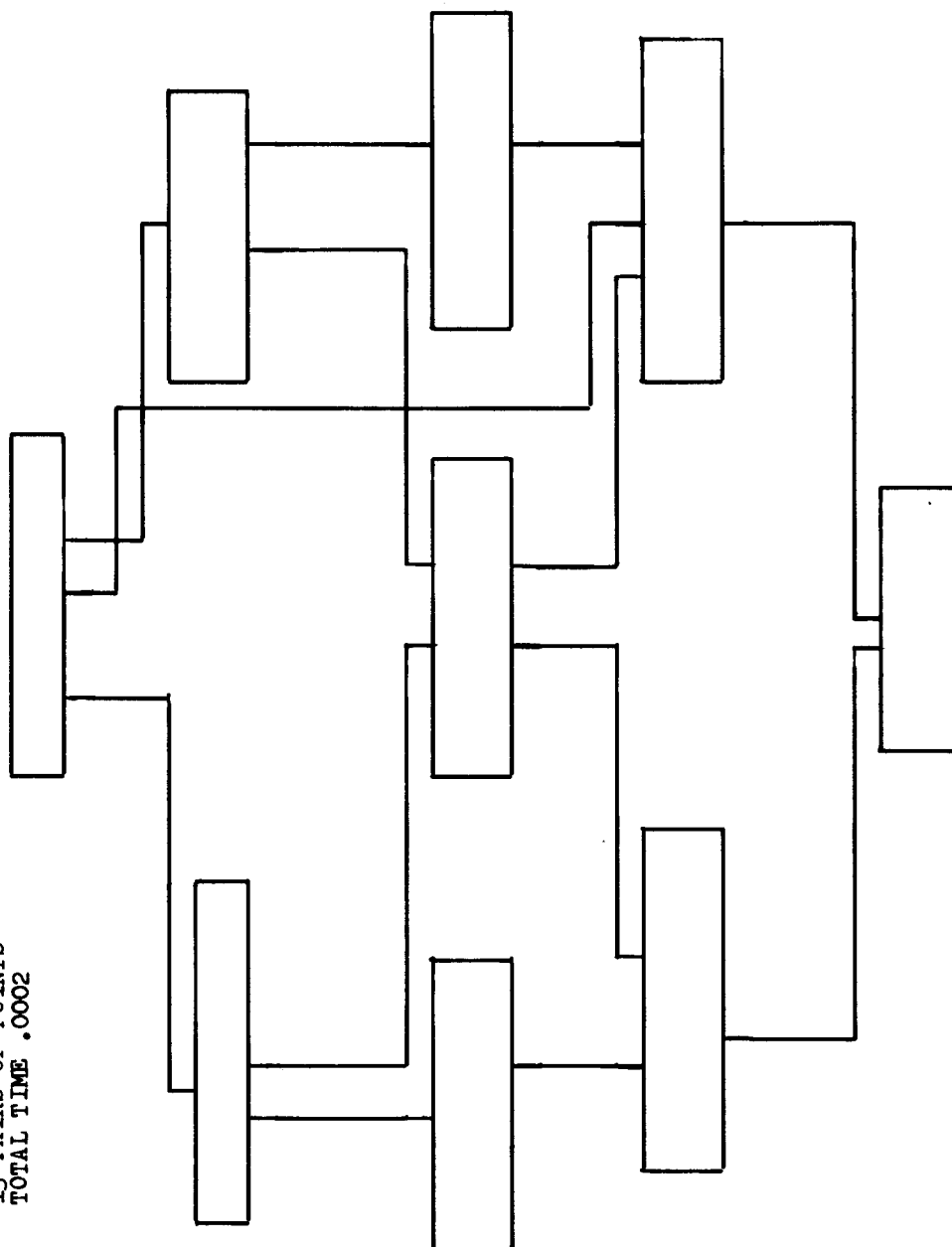


32 PAIRS OF POINTS
TOTAL TIME .0010

PRINTED CIRCUIT BOARD - II

PERT DIAGRAM

13 PAIRS OF POINTS
TOTAL TIME .0002



TWO SIDED BOARD

.0092 HRS

